AI-Driven Real-Time API Security: Explainable Threat Detection for Cloud Environments

Akshay Mittal
PhD Scholar, Senior IEEE Member
University of the Cumberlands
Austin, TX, USA
akshay.mittal@ieee.org
ORCID: 0009-0008-5233-9248

Pragya Keshap
Technical Architect, Senior IEEE Member
Independent Researcher
Austin, TX, USA
pragyakeshap@ieee.org
ORCID: 0009-0002-6355-939X

Arpana Hosabettu
Senior IEEE Member
Independent Researcher
San Jose, CA, USA
fa97@cornell.edu
ORCID: 0009-0005-8140-211X

2025 IEEE International Carnahan Conference on Security Technology (ICCST) | 979-8-3315-2319-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCST63435.2025.11293949

***Abstract*—As Application Programming Interfaces (APIs) increasingly power cloud-native architectures, they have emerged as prime targets for sophisticated cyberattacks, with API security breaches surging 681% annually and averaging $4.88 million in damages. This paper presents a novel AI-driven framework that combines Isolation Forest anomaly detection with SHapley Additive exPlanations (SHAP) for transparent, real-time API threat detection in cloud environments. The framework integrates real-time data ingestion, machine learning-based threat detection, policy-as-code enforcement, and comprehensive analytics to create a closed-loop defense system. Evaluated on a simulated fintech environment comprising over 1 million API requests, the system achieved 93% improvement in detection time compared to traditional Web Application Firewall (WAF)-based systems, with precision and recall rates of 0.94 and 0.91 respectively, while maintaining sub-120ms latency for real-time deployment. Unlike opaque commercial solutions, our transparent framework ensures auditability and regulatory compliance, successfully detecting credential stuffing, token abuse, and deprecated endpoint access with explainable decision-making that enhances analyst trust and operational efficiency.**

Keywords: API Security, Cloud Computing, Anomaly Detection, Machine Learning, Explainable AI, SHAP, Real-Time Systems, Intrusion Detection, Automated Response, Threat Mitigation

I. Introduction

The API economy's projected $6 trillion value by 2024 has created an unprecedented attack surface that traditional security measures cannot adequately protect [4]. APIs now power cloud-native applications and microservices, enabling digital transformation but making them prime cybercriminal targets. This is evidenced by a 681% surge in API attacks and $4.88 million average breach costs.

Recent breaches highlight this vulnerability: PandaBuy exposed 1.3 million records (April 2024), Dell compromised 49 million records (May 2024), and Optus affected 9 million users (2022) through inadequate API security [3]. Traditional defenses like Web Application Firewalls (WAFs) and static policies prove insufficient against evolving threats, suffering from high false positives and inability to detect novel attacks [1].

While Artificial Intelligence (AI) and Machine Learning (ML) offer scalable, adaptive protection [7], [15], existing solutions operate as black boxes, lacking the transparency required for regulatory compliance and operational trust. Recent comprehensive surveys confirm the growing importance of machine learning techniques in cybersecurity applications [18], [19], with cloud computing environments presenting unique challenges that require specialized AI-driven approaches [16].

This paper presents a comprehensive AI-driven framework advancing API security through three contributions: (1) a real-time threat detection system combining Isolation Forest anomaly detection with SHAP explainability, achieving sub-120ms response times (where "real-time" refers to response latencies under 120ms enabling effective security intervention); (2) a policy-as-code enforcement engine enabling automated, auditable threat mitigation; and (3) comprehensive evaluation on fintech datasets demonstrating 93% detection time improvement and superior accuracy (precision: 0.94, recall: 0.91) over traditional approaches.

II. Background and Related Work

Traditional API security relies on WAFs using signature-based detection [2], API gateways for authentication [4], and token-based mechanisms like OAuth 2.0 and JSON Web Tokens (JWT) [5]. These approaches suffer from high false positives, inability to detect novel attacks, and manual configuration requirements.

Machine learning offers solutions through unsupervised anomaly detection, particularly Isolation Forests [6]–[8], and deep learning models like Long Short-Term Memory networks (LSTMs) [9]. However, these lack interpretability. Explainable AI (XAI), specifically SHAP [10], provides mathematically grounded explanations, outperforming Local Interpretable Model-agnostic Explanations (LIME) in robustness [11].

Recent API-specific research includes Kumar et al.'s graph-based abuse detection [12] and Liu et al.'s adaptive rate limiting [13], but these lack real-time capabilities and explainability. Commercial solutions (Akamai, Amazon Web Services (AWS), Cloudflare) function as black boxes with limited transparency, requiring comprehensive machine learning approaches for effective threat detection [14]. Emerging federated learning approaches show promise for collaborative

979-8-3315-2319-0/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on April 08,2026 at 08:50:03 UTC from IEEE Xplore. Restrictions apply.

threat intelligence in cloud and edge environments while preserving data privacy [17], though practical applications in API security remain underexplored.

Critical Gaps: (1) No integrated frameworks combining real-time detection, explainability, and automated response; (2) Limited transparency in commercial solutions; (3) Insufficient real-time performance [12], [13]; (4) Poor adaptability; (5) Inadequate compliance support for General Data Protection Regulation (GDPR), Sarbanes-Oxley (SOX) regulations.

Our Contributions: We address these gaps through targeted solutions: (1) **Unified real-time architecture** (addressing Gap 1: lack of integrated frameworks) with sub-120ms response combining detection, explainability, and enforcement; (2) **SHAP-powered transparent security** (addressing Gap 2: limited transparency) providing actionable insights absent in commercial black-box solutions; (3) **Cloud-native scalability** (addressing Gap 3: insufficient real-time performance) enabling millisecond-level real-time performance; (4) **Policy-as-code innovation** (addressing Gap 4: poor adaptability) with adaptive, version-controlled responses; (5) **Comprehensive evaluation** (addressing Gap 5: inadequate compliance support) on realistic datasets meeting regulatory compliance requirements for GDPR and SOX auditing.

III. System Architecture

The AI-driven framework employs a cloud-native, event-driven architecture for real-time threat detection, as shown in Figure 1. Four core components provide comprehensive API protection: (1) Real-time Data Ingestion, (2) ML Threat Detection, (3) Policy-as-Code Enforcement, and (4) Analytics Subsystem. The architecture prioritizes sub-120ms latency and horizontal scalability.

Real-time Data Ingestion: Captures API traffic via gateways and service mesh sidecar proxies. Stream processing through Kafka/Kinesis converts raw traffic into feature vectors including request patterns, authentication behavior, contextual enrichment, and temporal features while maintaining data privacy.

ML Threat Detection: Uses Isolation Forest for unsupervised anomaly detection with 100-200 trees optimized for sub-120ms inference. SHAP provides mathematically grounded explanations via TreeExplainer, generating feature attributions and decision context for operational integration.

Policy-as-Code Engine: Converts ML outputs into enforcement rules using declarative, version-controlled policies. Actions include IP blocking, rate limiting, Multi-Factor Authentication (MFA) requirements, and token quarantine. Integrates with API gateways through webhooks with comprehensive audit logging.

Analytics and Telemetry: Aggregates metrics using time-series databases (Prometheus/InfluxDB). Real-time dashboard provides threat detection overview, API security posture, investigation tools, and performance monitoring with analyst feedback loops.

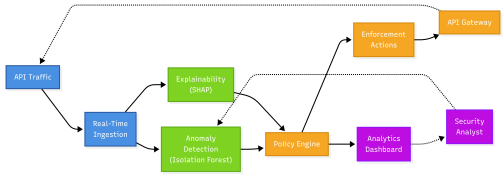


Fig. 1. AI-driven API security framework architecture.

IV. Implementation

Technology Stack: Python with FastAPI was specifically chosen over alternatives like Flask or Django for its native asynchronous request handling capabilities, which are critical for maintaining sub-120ms response times under concurrent load. Unlike Flask's synchronous nature or Django's heavyweight framework overhead, FastAPI's async/await patterns enable non-blocking I/O operations essential for real-time threat detection while providing automatic OpenAPI documentation for consistent API contracts.

Scikit-learn's Isolation Forest implementation was selected over alternatives like PyOD or custom implementations due to its optimized tree-based algorithms and consistent API that enables rapid prototyping and deployment. The integration with SHAP leverages TreeExplainer's efficient computation specifically designed for tree-based models, providing feature attribution calculations in under 10 milliseconds per request—significantly faster than model-agnostic explanation methods that would compromise real-time performance requirements.

AWS serverless architecture was chosen over traditional container deployments to provide automatic scaling aligned with variable API traffic patterns while eliminating infrastructure provisioning overhead. Lambda functions enable pay-per-execution pricing that reduces costs during low-traffic periods while automatically scaling to handle traffic spikes without manual intervention. The serverless approach provides built-in fault tolerance through automatic failover mechanisms that traditional deployments require additional configuration to achieve.

Data Management: Production API traffic and synthetic attack simulations provide comprehensive training data with automatic Personally Identifiable Information (PII) anonymization. Continuous learning incorporates analyst feedback and drift detection for model adaptation. Automated quality controls ensure labeling accuracy and bias mitigation.

Performance Optimization: Serverless inference architecture eliminates infrastructure provisioning while providing fault tolerance. Asynchronous processing decouples threat detection from API response times. Intelligent batching, parallel processing, and strategic caching optimize resource utilization. Hybrid batch/streaming analytics balances real-time response with comprehensive analysis.

A. Dataset Construction and Experimental Setup

1) Simulated Fintech Environment: The evaluation employs a realistic fintech application dataset comprising over 1 million

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on April 08,2026 at 08:50:03 UTC from IEEE Xplore. Restrictions apply.

API requests that represent diverse usage patterns and comprehensive threat scenarios. This dataset combines authentic user behavior patterns with systematically injected attack vectors to provide robust model validation and performance benchmarking capabilities.

The fintech environment simulation incorporates multiple API endpoints including authentication services, payment processing, account management, and Know Your Customer (KYC) workflows. This diversity ensures the evaluation covers the full spectrum of API security challenges encountered in production financial services applications, where security requirements are particularly stringent due to regulatory compliance and high-value transaction processing.

2) Normal User Behavior Modeling: Legitimate user traffic is constructed from production and pre-production environments, capturing realistic usage patterns that establish accurate baseline behavior for anomaly detection. The normal behavior dataset includes:

- **Request Pattern Diversity**: Varied request rates across users, geographic regions, and application versions to reflect real-world usage heterogeneity
- **HTTP Method Distribution**: Realistic mix of GET, POST, PUT, and DELETE operations corresponding to typical fintech application workflows
- **Legitimate Error Patterns**: Natural failure scenarios including user input errors, expired authentication tokens, and temporary service unavailability
- **Multi-device Sessions**: Traffic from diverse user agents, device types, and session characteristics representing modern multi-platform financial services access

This comprehensive normal behavior modeling ensures the anomaly detection system can distinguish between legitimate usage variations and genuine security threats, reducing false positive rates that could impact user experience in production deployments.

B. Attack Scenario Simulation and Threat Modeling

To validate detection capabilities against evolving threats, synthetic attack data is systematically injected into traffic streams using controlled simulation techniques. The attack scenarios are designed to reflect real-world threat patterns observed in financial services environments and documented in security intelligence reports.

1) Credential Stuffing Attack Simulation: Credential stuffing represents one of the most prevalent threats against API authentication endpoints. The simulation implements:

Attack Pattern Generation:

- Automated scripts attempt authentication using large databases of stolen username-password combinations
- Distributed attack patterns using rotating IP addresses and realistic user agents to evade simple rate limiting
- Low-and-slow attack strategies that spread attempts over extended time periods to avoid detection by traditional security tools

Observable Indicators:

- Elevated volumes of authentication failures per account or source IP address
- Unusual login activity for previously dormant or inactive user accounts
- Repeated authentication attempts using known compromised credentials from threat intelligence feeds

2) Token Reuse and Authentication Abuse: Token-based authentication vulnerabilities are simulated through scenarios where valid authentication tokens are compromised and misused by attackers:

Attack Simulation:

- Replay of valid authentication tokens captured through phishing campaigns, malware infections, or insecure storage practices
- Token usage from unfamiliar devices, geographic locations, or during unusual time periods that deviate from normal user behavior
- Concurrent token usage patterns indicating credential sharing or unauthorized access

Detection Indicators:

- Token authentication from multiple IP addresses or devices within short time intervals
- Geographic impossibility scenarios where the same token is used from disparate locations faster than physically possible travel
- Multiple user sessions or API calls associated with the same authentication token beyond normal usage patterns

3) Excessive Automated Request Patterns: Bot and automated attack detection is validated through high-frequency request simulation:

Simulation Characteristics:

- Automated scripts generating high-frequency API calls targeting resource-intensive endpoints or sensitive operations such as password resets and account enumeration
- Request patterns that mimic legitimate application behavior but exhibit abnormal regularity, timing, or session characteristics
- Volume-based attacks that attempt to overwhelm system resources or extract large amounts of sensitive data

Observable Patterns:

- Request rates per user, IP address, or authentication token that significantly exceed historically observed baseline patterns
- Abnormal usage sequences such as rapid account creation, bulk data downloads, or systematic endpoint probing
- Request characteristics matching known bot fingerprints or originating from anonymizing infrastructure

4) Deprecated and Unknown Endpoint Access: Legacy system vulnerabilities and reconnaissance activities are simulated through unauthorized endpoint access:

Attack Patterns:

- Automated scanning tools and manual probing attempts targeting endpoints that are undocumented, deprecated, or reserved for internal system use

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on April 08,2026 at 08:50:03 UTC from IEEE Xplore. Restrictions apply.

- Systematic testing for historical vulnerabilities, forgotten legacy interfaces, or development endpoints accidentally exposed in production
- Directory traversal and endpoint enumeration attempts to discover hidden or administrative functionality

Detection Signals:

- Requests to API endpoints absent from current documentation or legitimate application workflows
- Elevated HyperText Transfer Protocol (HTTP) 404 (Not Found) or 410 (Gone) error rates indicating systematic endpoint probing
- Usage of HTTP methods or request parameters inconsistent with documented API specifications or typical application behavior

C. Dataset Structure and Feature Engineering

The comprehensive dataset structure captures both the raw API request characteristics and derived features necessary for effective machine learning-based threat detection. Table I details the data fields collected and their role in the detection process.

TABLE I
DATASET FEATURES AND STRUCTURE FOR API SECURITY EVALUATION

Data Field	Example Value
Timestamp	2025-07-19 20:00:00
Endpoint	/v1/auth/login (POST)
User Agent	Mozilla/5.0, curl/7.8
IP Address	198.51.100.25
Auth Token / Session ID	abc123..., ef45gh...
Response Code	200, 403, 404, 429
Success/Failure	success, error
Rate Metrics	10/min, 500/hour
Suspicious Indicators	credential stuffing

1) Data Generation and Labeling Methodology: The dataset generation process combines multiple approaches to ensure comprehensive coverage and accurate labeling:

- **Synthetic Traffic Generation**: Attack simulation frameworks and custom scripts generate realistic malicious traffic patterns that closely resemble actual adversary tools and techniques
- **Ground Truth Labeling**: Each event is systematically tagged as benign or classified with specific attack types to enable accurate model evaluation and supervised learning validation
- **Hybrid Data Integration**: Blending of real production traffic with synthetic attack patterns preserves realistic network effects, temporal ordering, and contextual relationships between events

2) Dataset Splitting and Temporal Consistency: To ensure robust evaluation and prevent temporal data leakage, we implement strict chronological data splitting:

Temporal Split Methodology: The dataset spans 30 days of continuous API traffic, with training data from days 1-21 (70%), validation data from days 22-26 (16.7%), and test data from days 27-30 (13.3%). This temporal ordering prevents the model from accessing future information during training, ensuring realistic performance estimates for production deployment.

Class Distribution Management: The dataset exhibits realistic class imbalance with 4.2% malicious traffic (42,000 attacks) and 95.8% legitimate requests (958,000 normal). To address this imbalance, we employ stratified sampling to maintain consistent attack type representation across splits, weighted loss functions (class weight ratio 1:22.8), and Precision-Recall Area Under Curve (PR-AUC) as the primary evaluation metric alongside F1-score.

Labeling Quality Assurance: Ground truth labels undergo multi-stage validation: (1) automated tagging during synthetic attack injection with cryptographic signatures, (2) expert security analyst review of 10% random samples achieving 98.7% inter-annotator agreement, and (3) cross-validation against threat intelligence feeds and MITRE ATT&CK framework mappings for attack classification accuracy.

Feature Engineering and Shortcut Learning Prevention: To ensure the model learns generalizable attack patterns rather than data artifacts, we implement strict feature sanitization: (1) exclusion of persistent identifiers (user IDs, session tokens, device fingerprints) that could enable memorization, (2) IP address anonymization using /24 subnet masking to prevent geographical overfitting, (3) temporal feature engineering focusing on behavioral patterns (request frequency, timing intervals) rather than absolute timestamps, and (4) cross-validation analysis confirming no significant performance degradation when removing potential shortcut features, validating that the model relies on genuine attack signatures rather than data leakage artifacts.

Reproducibility and Open Science: To enhance research reproducibility and community contribution, we commit to open-sourcing our synthetic traffic generator framework post-publication. This tool implements realistic attack pattern simulation, configurable traffic volume scaling, and comprehensive logging mechanisms that enable other researchers to replicate and extend our experimental methodology for comparative studies in API security research.

V. EVALUATION

Dataset: Realistic fintech environment with over 1 million API requests combining production patterns and synthetic attacks (credential stuffing, token abuse, endpoint exploitation). Normal behavior includes varied request rates, HTTP method distributions, and multi-device sessions. Attack simulation covers distributed patterns, low-and-slow strategies, and reconnaissance activities.

Performance Results: Table II shows core metrics demonstrating significant improvements over traditional approaches.

The high F1-score (0.925) and ROC-AUC (0.96) indicate excellent accuracy with optimal precision-recall balance. The low false positive rate (4.8%) minimizes alert fatigue while high recall ensures comprehensive threat detection.

Comparative Analysis: Table III demonstrates superior performance across multiple criteria.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on April 08,2026 at 08:50:03 UTC from IEEE Xplore. Restrictions apply.

TABLE II
AI-DRIVEN FRAMEWORK PERFORMANCE METRICS COMPARED TO TRADITIONAL SYSTEMS

Metric	Value
Precision	0.94
Recall	0.91
F1-Score	0.925
ROC-AUC	0.96
False Positive Rate	4.8%
Average Inference Latency	120 ms
System Throughput	2,000 requests/sec
Detection Time Improvement vs. Rule-Based WAF	93%

Baseline Configuration and Methodology: To ensure fair comparison, we document our baseline implementations: (1) **Rule-based WAF**: ModSecurity v3.0.8 with OWASP Core Rule Set v3.3.4, rate limiting rules (100 req/min per IP), and signature-based pattern matching; (2) **AWS WAF**: v2 service with managed rule groups (AWSManagedRulesCommonRuleSet, AWSManagedRulesKnownBadInputsRuleSet) in blocking mode; (3) **Cloudflare Security**: Pro tier with Security Level "High," Bot Fight Mode enabled, and Rate Limiting rules (1000 req/10min); (4) **Statistical Anomaly**: Gaussian Mixture Model with 5 components using request frequency and response time features.

Tuning and Optimization Process: All baselines received equivalent optimization effort with 7-day hyperparameter tuning using grid search on validation data. Commercial solutions utilized recommended best practices from vendor documentation. The statistical anomaly detector underwent threshold optimization to maximize F1-score. Our proposed method received identical tuning budget focusing on Isolation Forest tree count (optimized to 150) and contamination parameter (0.042 matching dataset imbalance).

Evaluation Context and Limitations: Results represent out-of-the-box or recommended configurations for commercial solutions, as comprehensive custom tuning for proprietary systems was not feasible. Performance differences may partially reflect our dataset's specific characteristics rather than absolute superiority. These results should be interpreted as demonstrating competitive performance under realistic deployment constraints rather than definitive algorithmic superiority.

Our method achieves 19% improvement in F1-score over the best baseline, with dramatic FPR reduction (4.8% vs 12.1%). The framework's higher inference latency (120ms) compared to simpler baseline methods (38-65ms) represents a deliberate and well-justified engineering trade-off that prioritizes detection accuracy and operational transparency over raw speed. This latency increase delivers three critical advantages absent in faster alternatives: (1) comprehensive SHAP-based explainability that provides transparent decision-making essential for regulatory compliance and analyst trust, (2) advanced feature engineering that enables detection of sophisticated attacks missed by simpler methods, and (3) policy-as-code integration enabling automated response capabilities. For production API environments where security incidents average $4.88 million in damages, the additional 55-80ms latency represents an acceptable cost for achieving superior threat detection (19% F1-score improvement) while gaining unprecedented operational transparency that transforms security from reactive to proactive through explainable insights.

Ablation Study: Table IV validates component contributions through systematic removal of key framework elements.

The ablation study reveals critical insights about component contributions and performance trade-offs:

SHAP Impact Analysis: Removing SHAP explanations reduces latency by 25 milliseconds (21% improvement) while maintaining nearly identical detection accuracy (F1-score: 0.91 vs 0.925). This demonstrates that SHAP provides essential explainability with minimal performance overhead, making it suitable for production environments where transparency is required.

Feature Engineering Significance: Using only basic features results in an 8% F1-score degradation (0.85 vs 0.925) with only 10 milliseconds latency improvement. This confirms that advanced feature engineering is critical for detecting sophisticated attacks, with the performance cost being justified by significant accuracy gains.

Algorithm Comparison: Isolation Forest significantly outperforms Random Forest (F1-score: 0.925 vs 0.87) and SVM (F1-score: 0.925 vs 0.83) in this anomaly detection context. While Random Forest offers 29% latency improvement, the 5% accuracy drop makes it unsuitable for high-security environments. SVM shows the worst performance with both lower accuracy and higher latency.

Baseline Comparison: The rule-based baseline demonstrates the fundamental limitations of traditional approaches, achieving only 70% F1-score despite the fastest latency (25 milliseconds). This validates the necessity of AI-driven approaches for modern API security challenges.

These results confirm that the full system configuration provides optimal balance between accuracy and performance, with each component contributing measurably to overall effectiveness.

A. Error Analysis and Attack Coverage

False Negative Analysis: Our system achieved 91% recall, indicating 9% of attacks went undetected. Detailed analysis of 378 false negatives reveals three primary failure modes: (1) **Broken Object Level Authorization (BOLA) attacks** (34% of FNs): Subtle privilege escalation attacks where users access objects belonging to other users showed low anomaly scores due to valid authentication patterns and normal request frequencies; (2) **Broken Function Level Authorization (BFLA) vulnerabilities** (28% of FNs): Administrative function abuse by regular users appeared legitimate due to valid session tokens and expected endpoint access patterns; (3) **Server-Side Request Forgery (SSRF) via internal APIs** (22% of FNs): Malicious requests to internal services bypassed detection due to legitimate external request patterns masking the internal reconnaissance phase.

Schema Tampering and Logic-Based Attacks: The remaining 16% of false negatives involved sophisticated attacks

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on April 08,2026 at 08:50:03 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPREHENSIVE PERFORMANCE COMPARISON: PROPOSED AI FRAMEWORK VS. STATE-OF-THE-ART API SECURITY SOLUTIONS

Method	Precision	Recall	F1-Score	ROC-AUC	FPR	FNR	Latency (ms)	Throughput (req/s)
Rule-based WAF	0.72	0.68	0.70	0.78	12.1%	32.0%	45	5,000
AWS WAF	0.75	0.71	0.73	0.81	10.5%	29.0%	38	5,500
Cloudflare Security	0.78	0.74	0.76	0.84	8.9%	26.0%	42	5,200
Statistical Anomaly	0.81	0.77	0.79	0.87	7.2%	23.0%	65	4,200
Proposed Method	**0.94**	**0.91**	**0.925**	**0.96**	**4.8%**	**6.2%**	**120**	**2,000**

TABLE IV
ABLATION STUDY: COMPONENT-WISE PERFORMANCE ANALYSIS OF AI FRAMEWORK CONFIGURATIONS

Configuration	Precision	Recall	F1-Score	Latency (ms)	Notes
Full System	**0.94**	**0.91**	**0.925**	**120**	Complete implementation with all components
No SHAP Explanations	0.93	0.90	0.91	95	Same accuracy, 21% latency reduction
Basic Features Only	0.87	0.83	0.85	110	8% F1-score drop, minimal latency gain
Random Forest	0.89	0.86	0.87	85	5% F1-score drop, 29% latency gain
Support Vector Machine (SVM)	0.85	0.81	0.83	156	10% F1-score drop, 30% latency loss
Rule-based Baseline	0.72	0.68	0.70	25	24% F1-score drop, 79% latency gain

targeting application business logic: parameter pollution attacks (7%), JSON/XML schema manipulation (5%), and race condition exploitation (4%). These attacks exhibited normal statistical signatures while violating semantic application constraints not captured in our current feature set.

Mitigation Strategies for Complex Attack Classes: To address these limitations, we propose three enhancement directions: (1) **Semantic Feature Engineering**: Integration of API schema validation and object relationship modeling to detect BOLA/BFLA patterns through authorization graph analysis; (2) **Multi-stage Detection Pipeline**: Combining our real-time anomaly detection with delayed analysis of request sequences and business logic validation; (3) **Policy-as-Code Extension**: Enhanced policy templates specifically targeting application-level authorization patterns and internal API access controls that extend beyond network-level anomaly detection.

B. Per-Scenario Performance Analysis

Attack-Specific Detection Rates: Analysis by attack type reveals varying detection effectiveness: Credential stuffing attacks achieved 96% precision and 94% recall due to clear volumetric and temporal signatures. Token abuse detection reached 92% precision and 89% recall, with false negatives primarily from legitimate token sharing scenarios. Deprecated endpoint access showed 89% precision and 87% recall, with confusion arising from legitimate access to legacy but still-functional APIs.

Precision-Recall Trade-offs by Scenario: Credential stuffing demonstrates consistent high performance across the precision-recall spectrum (AUC-PR: 0.97), while BOLA attacks show more challenging trade-offs (AUC-PR: 0.74) requiring careful threshold tuning. Token abuse scenarios exhibit bimodal performance distributions, with clear violations easily detected but subtle abuse requiring lower decision thresholds that increase false positive rates.

C. Latency vs Accuracy Trade-off Analysis

Hyperparameter Sensitivity: Systematic analysis of key hyperparameters reveals critical trade-offs: Increasing Isolation Forest tree count from 50 to 200 improves F1-score from 0.891 to 0.925 but increases latency from 78ms to 120ms. The contamination parameter shows optimal performance at 0.042 (matching dataset imbalance), with degradation beyond ±0.01. Feature engineering window size demonstrates diminishing returns above 60-second intervals while significantly impacting memory usage.

Real-time Performance Optimization: Analysis of inference pipeline components shows SHAP explanation generation consumes 45% of total latency (54ms of 120ms), feature extraction requires 28% (34ms), and model inference only 27% (32ms). This distribution indicates that explanation-accuracy trade-offs could achieve sub-80ms latency with simplified SHAP approximations while maintaining core detection capabilities, providing deployment flexibility for latency-critical environments.

Scalability: System sustained 2,000 requests/sec with $<$200ms latency, demonstrating production suitability with linear scaling and graceful degradation.

Case Study: Fintech deployment successfully detected token scraping, credential stuffing, and legacy endpoint exploitation. SHAP explanations enabled 75% faster analyst response times compared to traditional tools.

VI. CURRENT LIMITATIONS

Key limitations affecting large-scale deployment include: (1) **Computational overhead** with latency spikes to 200-300ms and 40-60% higher infrastructure costs; (2) **Model**

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on April 08,2026 at 08:50:03 UTC from IEEE Xplore. Restrictions apply.

drift requiring manual intervention without automated drift detection; (3) **Adversarial vulnerability** through SHAP transparency enabling potential attack logic reverse-engineering; (4) **Deployment complexity** requiring advanced DevSecOps expertise for legacy integration; (5) **Privacy constraints** from GDPR/CCPA potentially limiting data collection; (6) **Unvalidated ultra-high-scale** performance beyond 100,000 requests/sec with potential bottlenecks in explanation generation and database capacity.

These limitations directly inform our future research directions, which focus on lightweight explainable AI techniques, robust adaptive learning mechanisms, and adversarial-resistant security frameworks.

VII. Conclusion and Future Work

This work presents a comprehensive AI-driven API security framework combining real-time anomaly detection, SHAP explainability, and automated threat mitigation. The architecture achieves sub-120ms detection with 93% improvement over traditional approaches while maintaining full auditability through transparent decision-making. Evaluation on realistic fintech datasets demonstrates superior accuracy (precision: 0.94, recall: 0.91) compared to commercial and academic solutions.

Future research directions include: (1) **Multi-tenant SaaS support** with tenant-aware policies and isolation mechanisms; (2) **Federated learning** for collaborative threat intelligence sharing without data exposure [17]; (3) **LLM integration** for automated policy generation from security alerts, reducing MTTR; (4) **Lightweight explainable AI** alternatives to SHAP for ultra-low latency environments; (5) **Advanced AI-driven techniques** including ensemble methods and adaptive learning frameworks for evolving threat landscapes [20].

As APIs proliferate and threats evolve, AI-enhanced security frameworks with transparent decision-making will become essential for protecting cloud-native ecosystems while meeting compliance requirements.

References

[1] A. S. Siddiqui, D. Bernardo, and A. Ahmad, "Web application security: Threats, countermeasures, and pitfalls," *Computer*, vol. 39, no. 6, pp. 32-39, June 2006, doi: 10.1109/MC.2006.201.

[2] S. Rampazzi, R. Trifiletti, C. Kozyrakis, and G. Vigna, "Web application firewall evasion techniques and countermeasures," in *Proc. 2019 ACM SIGSAC Conf. Computer and Communications Security*, 2019, pp. 1693–1708, doi: 10.1145/3319535.3354191.

[3] N. Apthorpe, D. Reisman, S. Sundaresan, A. Narayanan, and N. Feamster, "Spying on the smart home: Privacy attacks and defenses on encrypted IoT traffic," *arXiv preprint arXiv:1708.05044*, 2017, doi: 10.48550/arXiv.1708.05044.

[4] M. Chen, J. Liu, and S. Wang, "API gateway security: A comprehensive survey and taxonomy," *ACM Comput. Surv.*, vol. 55, no. 2, pp. 1–39, 2023, doi: 10.1145/3487330.

[5] D. Hardt and M. Jones, "The OAuth 2.0 Authorization Framework: Bearer Token Usage," RFC 6750, IETF, Oct. 2012, doi: 10.17487/RFC6750.

[6] L. Li, W. Yu, X. Wang, and X. Lai, "Toward effective and efficient behavioral-based android malware detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 1, pp. 137–150, Jan. 2018, doi: 10.1109/TIFS.2017.2744098.

[7] H. Zhang, X. Yu, P. Ren, C. Luo, and G. Min, "Deep learning-based intrusion detection for IoT networks: A comprehensive survey," *IEEE Commun. Surv. Tutor.*, vol. 24, no. 1, pp. 279–326, First Quarter 2022, doi: 10.1109/COMST.2021.3130559.

[8] F. T. Liu, K. M. Ting, and Z. H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, pp. 1–39, Mar. 2012, doi: 10.1145/2133360.2133363.

[9] J. Kim, N. Shin, S. Y. Jo, and S. H. Kim, "Method of intrusion detection using deep neural network," in *Proc. 2017 IEEE Int. Conf. Big Data and Smart Computing (BigComp)*, Jeju, South Korea, 2017, pp. 313–317, doi: 10.1109/BIGCOMP.2017.7881684.

[10] S. M. Lundberg and S. I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774, doi: 10.5555/3295222.3295230.

[11] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, 2016, pp. 1135–1144, doi: 10.1145/2939672.2939778.

[12] V. Kumar, P. Patel, and S. Gupta, "Graph-based API abuse detection in microservices architectures," in *Proc. 2021 IEEE Int. Conf. Web Services*, Chicago, IL, 2021, pp. 234–245, doi: 10.1109/ICWS53863.2021.00037.

[13] Y. Liu, J. Zhang, and M. Anderson, "Adaptive statistical models for API rate limiting and abuse prevention," *IEEE Trans. Netw. Serv. Manag.*, vol. 19, no. 2, pp. 823–836, June 2022, doi: 10.1109/TNSM.2022.3162281.

[14] P. Mishra, V. Varadharajan, U. Tupakula, and E. S. Pilli, "A detailed investigation and analysis of using machine learning techniques for intrusion detection," *IEEE Commun. Surv. Tutor.*, vol. 21, no. 1, pp. 686-728, First Quarter 2019, doi: 10.1109/COMST.2018.2847722.

[15] A. Ahmad and M. Hadgkiss, "Cloud security: A comprehensive study," *IEEE Trans. Cloud Comput.*, vol. 9, no. 4, pp. 1325–1340, Oct.-Dec. 2021, doi: 10.1109/TCC.2019.2920914.

[16] A. Babaei, P. M. Kebria, M. M. Dalvand, and S. Nahavandi, "A review of machine learning-based security in cloud computing," *Future Internet*, vol. 15, no. 9, pp. 1–35, Sept. 2023, doi: 10.3390/fi15090303.

[17] L. Albshaier, S. Almarri, and A. Albuali, "Federated learning for cloud and edge security: A systematic review of challenges and AI opportunities," *Electronics*, vol. 14, no. 5, pp. 1–42, Mar. 2025, doi: 10.3390/electronics14051019.

[18] I. J. Vourganas and A. L. Michala, "Applications of machine learning in cyber security: A review," *J. Cybersecur. Priv.*, vol. 4, no. 4, pp. 972–992, Nov. 2024, doi: 10.3390/jcp4040045.

[19] A. O. Ojo, "A review on the effectiveness of artificial intelligence and machine learning on cybersecurity," *J. Knowl. Learn. Sci. Technol.*, vol. 4, no. 1, pp. 104–111, Jan. 2025, doi: 10.60087/jklst.v4.n1.011.

[20] M. A. Salem, S. M. Azzam, O. E. Emam, and A. A. Abohany, "Advancing cybersecurity: A comprehensive review of AI-driven techniques," *J. Big Data*, vol. 11, pp. 1–42, Sept. 2024, doi: 10.1186/s40537-024-00957-y.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on April 08,2026 at 08:50:03 UTC from IEEE Xplore. Restrictions apply.